

Decisions Trees Have High Variance

Adding *variance* (i.e., *complexity*) is required for useful models.

Machine learning algorithms differ in the amount of variance that they can introduce.

Decision trees exhibit high *variance*.

Here's the intuition – trees can radically change their shape based on changes in the training data...

Dataset #1

occupation	relationship	income
Adm-clerical	Not-in-family	<=50K
Exec-managerial	Husband	<=50K
Handlers-cleaners	Not-in-family	<=50K
Handlers-cleaners	Husband	<=50K
Prof-specialty	Wife	<=50K
Exec-managerial	Wife	<=50K
Other-service	Not-in-family	<=50K
Exec-managerial	Husband	>50K
Prof-specialty	Not-in-family	>50K
Exec-managerial	Husband	>50K

Dataset #2

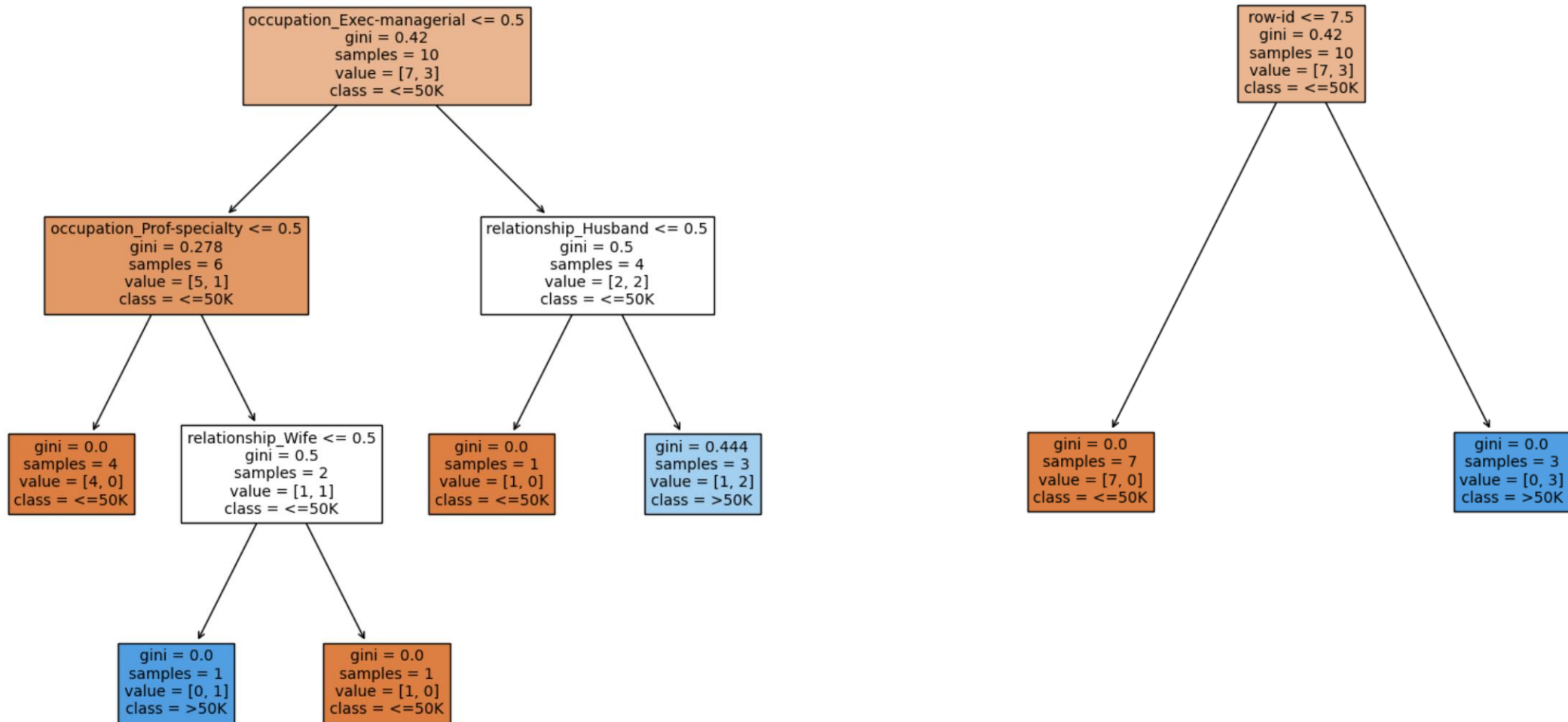
Only for teaching purposes!

row-id	occupation	relationship	income
1	Adm-clerical	Not-in-family	<=50K
2	Exec-managerial	Husband	<=50K
3	Handlers-cleaners	Not-in-family	<=50K
4	Handlers-cleaners	Husband	<=50K
5	Prof-specialty	Wife	<=50K
6	Exec-managerial	Wife	<=50K
7	Other-service	Not-in-family	<=50K
8	Exec-managerial	Husband	>50K
9	Prof-specialty	Not-in-family	>50K
10	Exec-managerial	Husband	>50K

High Variance Leads to Overfitting

By default, *scikit-learn* decision trees will grow to be very complex.

Depending on the data, this can produce “interesting” models...



Real World Trees

In practice, trees are rarely used for “Production” scenarios.

They can be useful for certain ad hoc analysis scenarios (e.g., explaining what’s going on in the business).

However, if one is planning on using a decision tree for decision-making, *optimal tuning is a must*.

In the real world, decision trees are wildly useful – but not on their own.

The high variance of decision trees has been leveraged to create one of the most useful of all ML algorithms...

The mighty random forest!

Wisdom of the Crowd

“The three conditions for a group to be intelligent are diversity, independence, and decentralization.” - James Surowiecki

ENSEMBLING:

Compiling predictions from multiple machine learning models in order to make more accurate predictions than any individual model.

However, ensembling works best when the individual models are diverse and have low correlation between their predictions.

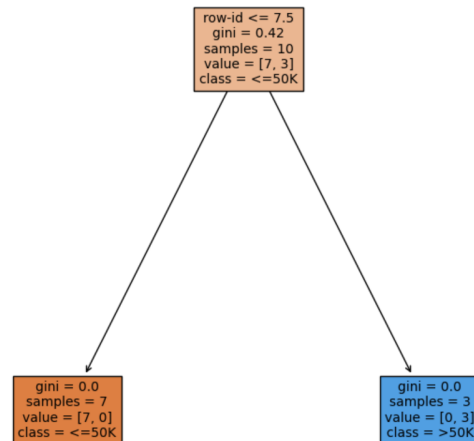
GOOD NEWS! The Random Forest algorithm manufactures independence and diversity across decision trees, making it an effective ensembling method.

Manufacturing Independence

The mighty random forest manufactures independence across decision trees via three mechanisms...

1 – Variance

row_id	occupation	relationship	income
1	Adm-clerical	Not-in-family	<=50K
2	Exec-managerial	Husband	<=50K
3	Handlers-cleaners	Not-in-family	<=50K
4	Handlers-cleaners	Husband	<=50K
5	Prof-specialty	Wife	<=50K
6	Exec-managerial	Wife	<=50K
7	Other-service	Not-in-family	<=50K
8	Exec-managerial	Husband	>50K
9	Prof-specialty	Not-in-family	>50K
10	Exec-managerial	Husband	>50K



2 - Bagging



Blue

Blue
Orange
Orange
Green

Blue
Orange
Orange
Green

Red
Green
Orange
Green

Orange
Red
Green
Blue

3 – Feature Randomization

	sex	occupation	work_class	relationship	income
1	Male	Adm-clerical	State-gov	Not-in-family	<=50K
2	Male	Exec-managerial	Self-emp-not-inc	Husband	<=50K
3	Male	Handlers-cleaners	Private	Not-in-family	<=50K
4	Male	Handlers-cleaners	Private	Husband	<=50K
5	Female	Prof-specialty	Private	Wife	<=50K
6	Female	Exec-managerial	Private	Wife	<=50K
7	Female	Other-service	Private	Not-in-family	<=50K
8	Male	Exec-managerial	Self-emp-not-inc	Husband	>50K
9	Female	Prof-specialty	Private	Not-in-family	>50K
10	Male	Exec-managerial	Private	Husband	>50K

	occupation	relationship	income
1	Adm-clerical	Not-in-family	<=50K
2	Exec-managerial	Husband	<=50K
3	Handlers-cleaners	Not-in-family	<=50K
4	Handlers-cleaners	Husband	<=50K
5	Prof-specialty	Wife	<=50K
6	Exec-managerial	Wife	<=50K
7	Other-service	Not-in-family	<=50K
8	Exec-managerial	Husband	>50K
9	Prof-specialty	Not-in-family	>50K
10	Exec-managerial	Husband	>50K

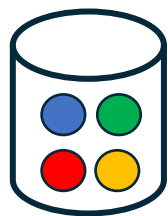
sex	work_class	income
1 Male	State-gov	<=50K
2 Male	Self-emp-not-inc	<=50K
3 Male	Private	<=50K
4 Male	Private	<=50K
5 Female	Private	<=50K
6 Female	Private	<=50K
7 Female	Private	<=50K
8 Male	Self-emp-not-inc	>50K
9 Female	Private	>50K
10 Male	Private	>50K

Randomizing Rows With Bagging

Trees will change their shape depending on the training data (i.e., *high variance*).

Bagging (i.e., “bootstrap aggregation”) takes a single training set and “manufactures” different datasets.

The magic of bagging works via *random sampling with replacement*...



Let's say I have a jar with 4 marbles – blue, green, red, and orange.



I reach into the jar and draw a marble at random.

List has as many items as marbles in the jar.

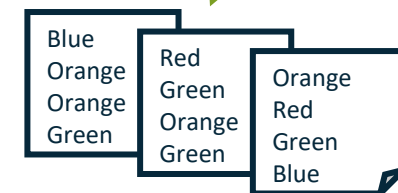


I write down the drawn marble and put it back in the jar.



I repeat this process, getting a list that is different from the jar.

Notice the duplicates!



I can create as many lists as I would like.

The marbles are the rows of your training set.

Bagging is Awesome

Bagging helps produce wildly different trees.

Bagging is also awesome because *bags don't have all the original training data...*

Bag 0

	sex	occupation	work_class	relationship	income
1	Male	Handlers-cleaners	Private	Husband	<=50K
2	Male	Handlers-cleaners	Private	Husband	<=50K
3	Male	Handlers-cleaners	Private	Husband	<=50K
4	Female	Other-service	Private	Not-in-family	<=50K
5	Female	Prof-specialty	Private	Not-in-family	>50K
6	Female	Prof-specialty	Private	Not-in-family	>50K
7	Male	Exec-managerial	Private	Husband	>50K
8	Female	Prof-specialty	Private	Not-in-family	>50K
9	Female	Prof-specialty	Private	Not-in-family	>50K
10	Male	Handlers-cleaners	Private	Husband	<=50K

Bag 1

	sex	occupation	work_class	relationship	income
1	Male	Adm-clerical	State-gov	Not-in-family	<=50K
2	Male	Adm-clerical	State-gov	Not-in-family	<=50K
3	Male	Handlers-cleaners	Private	Not-in-family	<=50K
4	Male	Exec-managerial	Self-emp-not-inc	Husband	>50K
5	Male	Exec-managerial	Self-emp-not-inc	Husband	<=50K
6	Female	Prof-specialty	Private	Not-in-family	>50K
7	Male	Exec-managerial	Self-emp-not-inc	Husband	>50K
8	Female	Prof-specialty	Private	Not-in-family	>50K
9	Male	Handlers-cleaners	Private	Husband	<=50K
10	Male	Adm-clerical	State-gov	Not-in-family	<=50K

Out of bag (OOB) data

	sex	occupation	work_class	relationship	income
1	Male	Adm-clerical	State-gov	Not-in-family	<=50K
2	Male	Exec-managerial	Self-emp-not-inc	Husband	<=50K
3	Male	Handlers-cleaners	Private	Not-in-family	<=50K
4	Female	Prof-specialty	Private	Wife	<=50K
5	Female	Exec-managerial	Private	Wife	<=50K
6	Male	Exec-managerial	Self-emp-not-inc	Husband	>50K

Out of bag (OOB) data

	sex	occupation	work_class	relationship	income
1	Female	Prof-specialty	Private	Wife	<=50K
2	Female	Exec-managerial	Private	Wife	<=50K
3	Female	Other-service	Private	Not-in-family	<=50K
4	Male	Exec-managerial	Private	Husband	>50K

Feature Randomization

The mighty random forest doesn't stop with bagging!

Random subsets of the features are provided to the decision trees...

Features

Label

	age	work_class	fnlwgt	education	education_num	marital_status	occupation	relationship	race	sex	capital_gain	capital_loss	hours_per_week	native_country	income
1	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
2	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	<=50K
3	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	<=50K

By default, the *RandomForestClassifier* (RF) chooses the square root of the number of features at random:

$$\sqrt{14} = 3.7417 = 3$$

Only use the whole part.

Let's say that the RF is building a new tree from scratch.

The decision tree then picks the best split given the 3 features.

The RF algorithm chooses 3 features at random to be considered for the first split:

	fnlwgt	relationship	hours_per_week	income
1	77516	Not-in-family	40	<=50K
2	83311	Husband	13	<=50K
3	215646	Not-in-family	40	<=50K

Feature Randomization

Continuing this hypothetical example...

The RF algorithm picks these 3 random features for the first split of the tree.

	fnlwgt	relationship	hours_per_week	income
1	77516	Not-in-family	40	<=50K
2	83311	Husband	13	<=50K
3	215646	Not-in-family	40	<=50K

The tree picks the best split of the 3 features.

	age	education_num	marital_status	income
1	39	13	Never-married	<=50K
2	50	13	Married-civ-spouse	<=50K
3	38	9	Divorced	<=50K

The RF algorithm picks 3 more features at random for the left node split.

The tree picks the best split of the 3 features.

	relationship	sex	capital_gain	income
1	Not-in-family	Male	2174	<=50K
2	Husband	Male	0	<=50K
3	Not-in-family	Male	0	<=50K

The RF algorithm picks 3 more features at random for the right node split.

The tree picks the best split of the 3 features.

Feature Randomization

Continuing this hypothetical example...

The RF algorithm picks these 3 random features for the first split of the tree.

	fnlwgt	relationship	hours_per_week	income
1	77516	Not-in-family	40	<=50K
2	83311	Husband	13	<=50K
3	215646	Not-in-family	40	<=50K

The tree picks the best split of the 3 features.

	age	education_num	marital_status	income
1	39	13	Never-married	<=50K
2	50	13	Married-civ-spouse	<=50K
3	38	9	Divorced	<=50K

The RF algorithm picks 3 more features at random for the left node split.

The tree picks the best split of the 3 features.

	relationship	sex	capital_gain	income
1	Not-in-family	Male	2174	<=50K
2	Husband	Male	0	<=50K
3	Not-in-family	Male	0	<=50K

The RF algorithm picks 3 more features at random for the right node split.

The tree picks the best split of the 3 features.

The Dataset

```
import pandas as pd

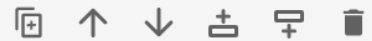
adult_train = pd.read_csv('adult_train.csv')
adult_train.head()
```

	Age	WorkClass	Fnlwgt	Education	EducationNum	MaritalStatus	Occupation	Relationship	Race	Sex	CapitalGain	CapitalLoss
0	20	Private	30035	Assoc-acdm	12	Never-married	Sales	Own-child	White	Female	0	0
1	22	Private	141297	Some-college	10	Never-married	Tech-support	Own-child	White	Male	0	0
2	63	Private	440607	Preschool	1	Married-civ-spouse	Prof-specialty	Husband	Other	Male	0	0
3	32	Private	165949	Bachelors	13	Divorced	Machine-op-inspct	Not-in-family	White	Female	0	1590
4	36	Private	139743	Some-college	10	Widowed	Prof-specialty	Own-child	White	Female	0	0

The Dataset

```
import pandas as pd
```

```
adult_train = pd.read_csv('adult_train.csv')  
adult_train.head()
```



	WorkClass	Fnlwgt	Education	EducationNum	MaritalStatus	Occupation	Relationship	Race	Sex	CapitalGain	CapitalLoss	HoursPerWeek	NativeCountry	Label
0	Private	30035	Assoc-acdm	12	Never-married	Sales	Own-child	White	Female	0	0	40	United-States	<=50K
1	Private	141297	Some-college	10	Never-married	Tech-support	Own-child	White	Male	0	0	40	United-States	<=50K
2	Private	440607	Preschool	1	Married-civ-spouse	Prof-specialty	Husband	Other	Male	0	0	30	Mexico	<=50K
3	Private	165949	Bachelors	13	Divorced	Machine-op-inspct	Not-in-family	White	Female	0	1590	42	United-States	<=50K
4	Private	139743	Some-college	10	Widowed	Prof-specialty	Own-child	White	Female	0	0	40	United-States	<=50K

[illegible]

Preparing Features

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 15016 entries, 0 to 15015
```

```
Data columns (total 32 columns):
```

#	Column	Non-Null Count	Dtype
0	Age	15016 non-null	int64
1	EducationNum	15016 non-null	int64
2	Female	15016 non-null	int64
3	CapitalGain	15016 non-null	int64
4	CapitalLoss	15016 non-null	int64
5	HoursPerWeek	15016 non-null	int64
6	MaritalStatus_Divorced	15016 non-null	bool
7	MaritalStatus_Married-AF-spouse	15016 non-null	bool
8	MaritalStatus_Married-civ-spouse	15016 non-null	bool
9	MaritalStatus_Married-spouse-absent	15016 non-null	bool
10	MaritalStatus_Never-married	15016 non-null	bool
11	MaritalStatus_Separated	15016 non-null	bool
12	MaritalStatus_Widowed	15016 non-null	bool

Preparing Labels

```
from sklearn.preprocessing import LabelEncoder

# Encode labels
label_encoder = LabelEncoder()
adult_y = label_encoder.fit_transform(adult_train['Label'])

print(label_encoder.classes_)
print(adult_y)

['<=50K' '>50K']
[0 0 0 ... 1 1 1]
```

Training a Mighty Random Forest

```
from sklearn.ensemble import RandomForestClassifier

# Instantiate the Random Forest object
adult_rf = RandomForestClassifier(n_estimators = 200, oob_score = True,
                                n_jobs = 2, random_state = 12345)

# Train the RandomForestClassifier
adult_rf.fit(adult_X, adult_y)

# What is the accuracy estimate?
print(f'Estimated accuracy with OOB data: {adult_rf.oob_score_: .4f}')
```

```
# What is the accuracy on the training data?
print(f'Training data accuracy: {adult_rf.score(adult_X, adult_y): .4f}')
```

```
Estimated accuracy with OOB data: 0.8135
Training data accuracy: 0.9638
```

Get OOB Predictions

```
# Get the out-of-bag (OOB) predictions as a DataFrame
oob_preds = pd.DataFrame(adult_rf.oob_decision_function_,
                          columns = label_encoder.classes_)

# Create a new column for the Label prediction
oob_preds['Label'] = 0

# Set Label for >50K if a prediction is greater than 50%
oob_preds.loc[oob_preds['>50K'] > 0.5, 'Label'] = 1

print(oob_preds.shape)
print(oob_preds.head(n = 10))
```

```
(15016, 3)
```

	<=50K	>50K	Label
0	1.000000	0.000000	0
1	0.987654	0.012346	0
2	0.666667	0.333333	0
3	0.780952	0.219048	0
4	0.865854	0.134146	0
5	0.376471	0.623529	1
6	0.971831	0.028169	0
7	1.000000	0.000000	0
8	0.840000	0.160000	0
9	0.606838	0.393162	0

Visualize OOB Predictions

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Visualize OOB predictions using a confusion matrix
cm = confusion_matrix(adult_y, oob_preds['Label'])
cmd = ConfusionMatrixDisplay(cm, display_labels = label_encoder.classes_)
cmd.plot();
```

Visualize OOB Predictions

